МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ
Ордена Трудового Красного Знамени федеральное государственное
Бюджетное образовательное учреждение высшего образования
МОСКОВСКИЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ СВЯЗИ И ИНФОРМАТИКИ

Кафедра «Программная инженерия»

Лабораторная работа № 3
по дисциплине «Структуры и алгоритмы обработки данных»
по теме: «Экспериментальное сравнение простых сортировок; QuickSort с рандомизацией опорного элемента»

Выполнил студент группы:
БВТ2505
Соронина Арина Андреевна
Проверил: Лапин Виктор Андреевич

Москва 2026

Цель и задачи работы:

Цель данной лабораторной работы: Освоить простые сортировки и QuickSort, экспериментально подтвердить их асимптотику на разных классах входов и понять роль стабильности и рандомизации опорного элемента.

Задачи:

1. Реализовать сортировки пузырьком, вставками и выбором со счётчиками сравнений и обменов (перемещений); счётчики выводятся вместе с результатом.
2. Реализовать QuickSort с рандомизированным выбором опорного элемента (random.randrange); рекурсию ограничить корректной обработкой малых подмассивов.
3. Сгенерировать по варианту три класса входов — упорядоченный, случайный, обратно упорядоченный — не менее 5 размеров с геометрическим шагом и выполнить замеры по методике бенчмаркинга (прогрев, ≥5 повторов, медиана).
4. Продемонстрировать стабильность/нестабильность алгоритмов на массиве пар «(ключ, метка)»: показать, какие сортировки сохраняют исходный порядок меток при равных ключах, а какие — нет.
5. Построить log-log графики времени от размера входа для всех алгоритмов и классов входов; сопоставить наклоны с $O(n^2)$ и $O(n \log n)$, объяснить различия между классами входов (в т. ч. лучший случай вставок и поведение пузырька на упорядоченных данных).
6. Верифицировать реализации: проверить инварианты сортировки (результат упорядочен и является перестановкой входа) и выполнить сверку с sorted() на нескольких сотнях случайных входов; граничные случаи — пустой массив, один элемент, все элементы равны, уже отсортированный и обратно отсортированный массивы.

Выполнение

1. Реализация

Все четыре сортировки работают с копией входа и не меняют исходный список; простые сортировки возвращают кортеж (результат, число сравнений, число обменов), так что счётчики выводятся вместе с результатом.

Алгоритм	**Лучший случай**	**Средний**	**Худший**	**Доп. память**	**Стабильна**
Пузырёк (с досрочным выходом)	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$	$O(1)$	да
Вставки	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$	$O(1)$	да
Выбор	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$	$O(1)$	нет
QuickSort, случайный опорный	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n^2)$	$O(\log n)$	нет

Пузырёк. Если за проход не было ни одного обмена, цикл прекращается: на упорядоченном входе это один проход и n - 1 сравнений. Обмен только при строгом a[j] > a[j+1], поэтому равные элементы не переставляются.

Вставки. Элемент сдвигает вправо все большие элементы префикса; третий счётчик − число сдвигов. Время Θ(n + число инверсий), сдвиг только при строгом >.

Выбор. Внутренний цикл всегда просматривает весь неотсортированный остаток, поэтому сравнений ровно n(n - 1)/2 на любом входе; обменов не больше n - 1 (обмен с самим собой не выполняется и не считается).

QuickSort. Опорный элемент выбирается как a[rng.randrange(lo, hi + 1)] (границы включительные, правую границу randrange не включает); генератор rng передаётся снаружи, поэтому запуск воспроизводим по seed. Разбиение трёхчастное (< p, = p, > p): равные опорному элементы остаются на месте и в

рекурсию не идут. Рекурсия заменена циклом со стеком отрезков: отрезки из 0 или 1 элемента не обрабатываются, а в стек откладывается большая часть, поэтому глубина стека не превышает $\log_2 n$.

2. Счётчики операций

Число обменов пузырька и сдвигов вставок ровно равно числу инверсий во входе, а у выбора число сравнений не зависит от данных. Таблица получена запуском с --variant 19 (seed = 49), n = 1000; в ячейках − сравнений / обменов (у вставок − сравнений / сдвигов).

Тип входа	**Инверсии**	**Пузырёк: сравнения / обмены**	**Вставки: сравнения / сдвиги**	**Выбор: сравнения / обмены**
упорядоченный	0	999 / 0	999 / 0	499 500 / 0
случайный	248 720	498 759 / 248 720	249 710 / 248 720	499 500 / 992
обратный	499 500	499 500 / 499 500	499 500 / 499 500	499 500 / 500

Упорядоченный вход. Пузырёк и вставки делают по 999 = n - 1 сравнений и ни одного обмена (лучший случай $\Theta(n)$); выбор − всё равно 499 500 = n(n - 1)/2.

Случайный вход. Инверсий 248 720 − близко к ожидаемым n(n - 1)/4 = 249 750. У вставок 249 710 = 248 720 + 990: сравнений столько же, сколько сдвигов, плюс по одной остановке на каждом шаге, кроме тех, где элемент дошёл до начала массива. Пузырёк делает в два раза больше сравнений, чем вставки (498 759 против 249 710, то есть $\approx n^2/2$ против $\approx n^2/4$); число сравнений пузырька ниже n(n - 1)/2 = 499 500 благодаря досрочному выходу. Выбор сделал 992 обмена ($\leq n - 1 = 999$).

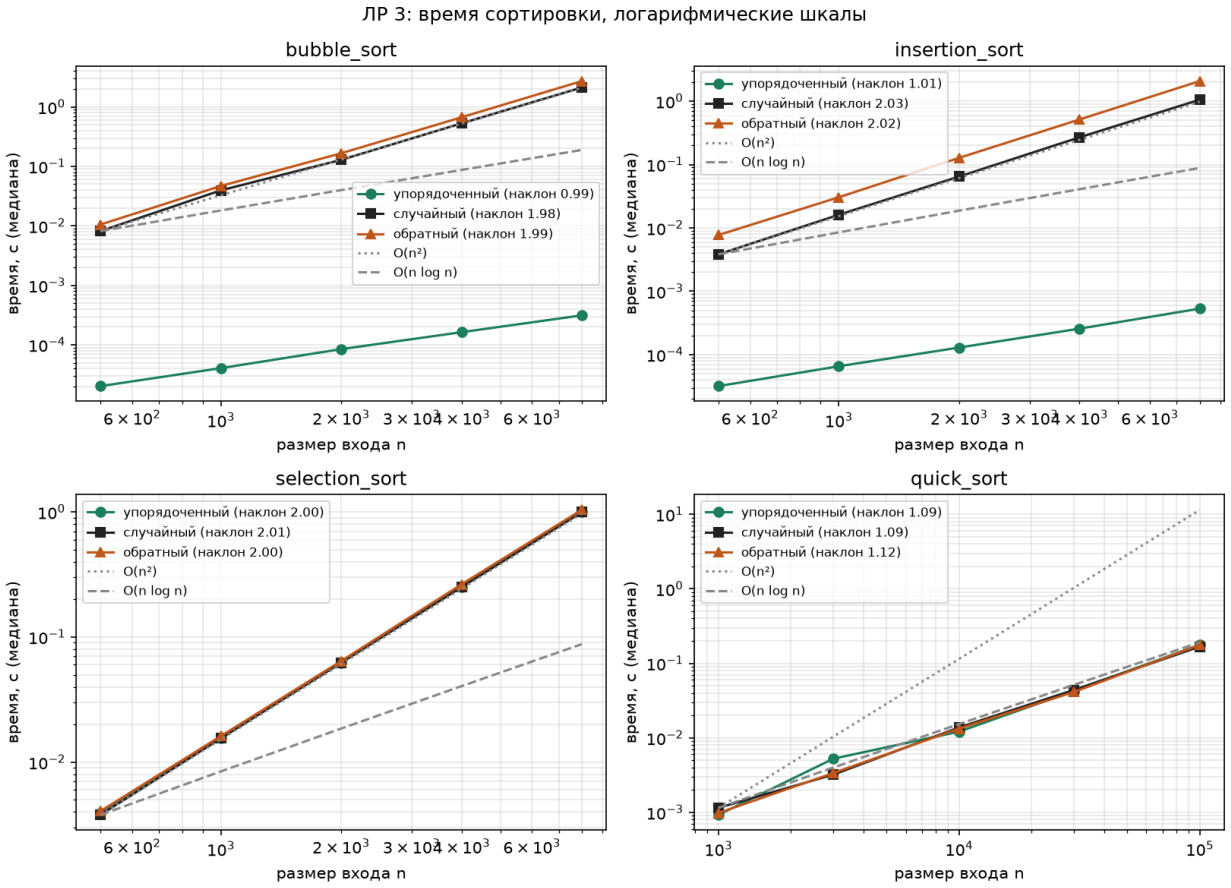
Обратный вход. Инверсий ровно n(n - 1)/2 = 499 500 (все значения разные), и пузырёк с вставками делают по 499 500 сравнений и обменов

(сдвигов) − худший случай. У выбора всего 500 = n/2 обменов: каждый ставит на место сразу два элемента (минимум и максимум).

Вывод. Пузырёк и вставки меняют только соседей и убирают по одной инверсии за обмен, поэтому им нужно не меньше обменов, чем инверсий (в среднем $n(n - 1)/4 − \Theta(n^2)$); выбор меняет далёкие элементы, поэтому его обменов очень мало, но сравнений − всегда максимум.

3. Замеры времени и наклоны

Время простых сортировок растёт как n^2 на случайном и обратном входах (наклон ≈ 2 в log-log осях), QuickSort − почти линейно, как n log n (наклон 1,09–1,12), причём на всех трёх классах входов.



Серые опорные линии O(n^2) и O(n log n) проведены через точку случайного входа при наименьшем n. Наклон − наклон прямой, построенной методом наименьших квадратов по всем пяти размерам: если $t \approx c \cdot n^k$, то наклон равен k.

Алгоритм	Время при макс. n, с: упоряд.	случайн.	обратн.	Наклон: упоряд.	случайн.	обратн.
Пузырёк (n = 8000)	0,0003	2,12	2,71	0,99	1,98	1,99
Вставки (n = 8000)	0,0005	1,05	2,07	1,01	2,03	2,02
Выбор (n = 8000)	1,00	0,99	1,04	2,00	2,01	2,00
QuickSort (n = 100 000)	0,178	0,168	0,177	1,09	1,09	1,12

Почему классы входов различаются:

1. **Пузырёк на упорядоченных данных − наклон ≈ 1 (Θ(n)):** один проход без обменов и досрочный выход благодаря флагу swapped. Без флага он остался бы квадратичным и на упорядоченных данных. На обратном входе он дольше, чем на случайном (2,71 с против 2,12 с), потому что обменов там вдвое больше (499 500 против 248 720 при n = 1000).

2. **Вставки − лучший случай.** На упорядоченном входе наклон ≈ 1: n - 1 сравнений и ноль сдвигов, время Θ(n + число инверсий) = Θ(n). На случайном входе инверсий в два раза меньше, чем на обратном, поэтому и время вдвое меньше (1,05 с против 2,07 с).

3. **Выбор** даёт наклон ≈ 2 и почти одинаковое время на всех трёх классах (около 1 с при n = 8000): число сравнений не зависит от данных, а обменов мало.

4. **QuickSort.** Наклон 1,09–1,12 − это не линия с наклоном 1 и не парабола с наклоном 2: для $t = c \cdot n \cdot \ln n$ локальный наклон равен $1 + 1/\ln n$, то есть около 1,09–1,14 на диапазоне 10^3–10^5, что и наблюдается. Упорядоченный и обратный входы не деградируют в квадрат: случайный опорный не даёт плохому входу влиять на выбор, разница между классами при n = 100 000 не превышает 6 %.

5. **Расхождения с теорией.** На графике QuickSort есть выброс при n = 3000 на упорядоченном входе (5,2 мс против 3,2 мс на случайном); вероятно, это шум замера на малых временах (медиана из пяти запусков), а не свойство алгоритма; на наклон он влияет слабо. Абсолютный масштаб: QuickSort сортирует 100 000 элементов за 0,17 с, а пузырёк − в 12,5 раза меньший массив из 8000 за 2,1 с.

4. Стабильность

Пузырёк и вставки сохраняют порядок меток при равных ключах, выбор и QuickSort − нет. Для демонстрации сортируются пары (ключ, метка), причём сравнение идёт только по ключу (класс Pair переопределяет <, >, <=, >= по ключу). Вход: (2, a), (1, b), (2, c), (1, d).

Сортировка	**Результат**	**Метки при ключе 1**	**Метки при ключе 2**	**Стабильна**
Пузырёк	(1, b), (1, d), (2, a), (2, c)	b, d	a, c	да
Вставки	(1, b), (1, d), (2, a), (2, c)	b, d	a, c	да
Выбор	(1, b), (1, d), (2, c), (2, a)	b, d	c, a	нет
QuickSort	(1, d), (1, b), (2, c), (2, a)	d, b	c, a	нет

Единственный пример не доказывает стабильность: QuickSort мог бы «повезти». Поэтому демонстрация дополнительно ищет нарушение среди 300 случайных массивов с ключами 0..4 (метка − исходный номер, эталон − стабильный sorted()): у пузырька и вставок нарушений нет, а у выбора и QuickSort находится контрпример. Например, QuickSort (с rng = Random(0)) на входе (4,0), (0,1), (2,2), (0,3), (3,4) даёт (0,3), (0,1), (2,2), (3,4), (4,0): метки при ключе 0 поменялись местами.

- **Пузырёк и вставки стабильны**, потому что меняют только соседей (сдвигают элемент) и только при строгом >: элемент не перепрыгивает через равный. Замена > на >= сломала бы стабильность.
- **Выбор нестабилен:** обмен дальнего минимума с a[i] перепрыгивает через равные ключи (в примере (2, a) уходит за (2, c)).
- **QuickSort нестабилен:** разбиение переставляет элементы на большие расстояния.
- **Где стабильность нужна:** при сортировке по нескольким ключам: сначала по второстепенному (например, фамилия), затем стабильно по главному (группа) − внутри каждой группы фамилии останутся по алфавиту.

5. *QuickSort: средний и худший случай, роль рандомизации*

Со случайным опорным QuickSort делает в среднем $\Theta(n \log n)$ операций на любом входе, а квадрат возможен только как маловероятный исход бросков. На наших замерах все три класса входов дают одинаковое время при n = 100 000 (0,168–0,178 с).

a. Разбиение.

После выбора опорного отрезок делится на части «< p», «= p», «> p» за близкое к n число сравнений, то есть за $\Theta(n)$. Работа на одном уровне рекурсии не превышает n.

b. Хорошее разбиение (пополам).

$T(n) = 2T(n/2) + \Theta(n)$ даёт $\Theta(n \log n)$ по мастер-теореме (второй случай): $\log_2 n$ уровней по $\leq n$ работы. Даже если каждое разбиение делит отрезок как 1 : 9, уровней всё равно $O(\log n)$ ($\approx 6{,}6 \cdot \log_2 n$); любая фиксированная пропорция 1 : q даёт тот же порядок.

c. Средний случай.

Каждая пара элементов $z_i < z_j$ сравнивается не более одного раза и только если первым опорным из $z_i \ldots z_j$ стал именно z_i или z_j; вероятность $2/(j - i + 1)$. Суммарно ожидаемое число сравнений − $2(n+1) \cdot H(n) - 4n \approx 2n \cdot \ln n \approx 1{,}39 \cdot n \cdot \log_2 n$.

d. Худший случай.

Если каждое разбиение отрезает $O(1)$ элементов (опорный − почти минимум или максимум), $T(n) = T(n - 1) + \Theta(n)$ и всего $(n - 1) + (n - 2) + \ldots + 1 = n(n - 1)/2 = \Theta(n^2)$ сравнений, глубина рекурсии − n.

e. Роль рандомизации.

С фиксированным опорным (первым или последним) упорядоченный и обратный входы − это как раз худший случай, причём один из самых частых на практике, и глубина рекурсии может дойти до n (в Python − RecursionError). Случайный опорный переносит случайность внутрь алгоритма, поэтому оценка ожидаемая для любого входа: плохой вход подобрать нельзя, неудачными бывают только броски. Худший случай остаётся теоретически возможным, но для него нужно много раз подряд вытянуть почти крайний элемент.

f. Равные ключи и стек.

Разбиение Ломуто на массиве из равных элементов дало бы $\Theta(n^2)$ даже со случайным опорным; трёхчастное разбиение выбрасывает равные за один проход. Рекурсия по меньшей части гарантирует глубину стека не больше $\log_2 n$, только память меняется, время − нет.

6. Верификация

Все восемь блоков self_check проходят, и замеры запускаются только после их прохождения. Проверки написаны как явные expect (не assert, который исчезает при python -O).

Проверка	**Что проверяется**
Инварианты (каждая из 4 сортировок)	результат — список, упорядочен (res[i] <= res[i+1]) и является перестановкой входа (sorted(res) == sorted(a))
Граничные входы	пустой массив, один элемент, все равны [5,5,5,5], упорядоченный [1..5], обратный [5..1]
Сверка с sorted()	300 случайных входов длиной 0–80 со значениями от -100 до 100 (встречаются повторы)
Счётчики выбора	ровно n(n - 1)/2 сравнений на любом входе
Счётчики пузырька и вставок	n = 5: упорядоченный — (n - 1, 0), обратный — (n(n - 1)/2, n(n - 1)/2); пример лекции [5, 2, 4, 6, 1, 3]: вставки 12 / 9, пузырёк 15 / 9, выбор 15 / 3
Обмены = инверсии	на 200 случайных массивах число обменов пузырька и сдвигов вставок равно числу инверсий, посчитанному перебором пар
Вход не меняется	список, переданный сортировке, остаётся прежним
Стабильность	пузырёк и вставки стабильны на 300 случайных массивах пар

Вывод

В ходе лабораторной работы были исследованы четыре алгоритма сортировки и сопоставлены их теоретические оценки с экспериментальными данными. Пузырьковая сортировка и сортировка вставками показали линейное время на упорядоченном массиве и квадратичное – на случайном и обратном. Сортировка выбором во всех случаях продемонстрировала квадратичный рост времени. QuickSort обрабатывал массивы существенно большего размера при сопоставимом времени для разных типов входа; измеренные наклоны согласуются с отсутствием сильной зависимости от исходного порядка в проведённом эксперименте. Счётчики подтвердили связь числа обменов пузырьковой сортировки и сдвигов сортировки вставками с числом инверсий, а также фиксированное число сравнений сортировки выбором. Проверки корректности и стабильности позволяют подтвердить соответствие реализаций заявленным свойствам.